Understanding Hooks in React

React Hooks are functions that allow you to use **state** and other **React features** in functional components. Before hooks, managing state or side effects in React was only possible with **class components**. Hooks provide a simpler and more readable approach for using state, context, refs, and other React features in functional components.

The useState() hook is one of the most commonly used hooks in React. It lets you add and manage state (data) inside a functional component. Other Hooks include useEffect, useNavigate, useContext() etc.

1. Introduction to useState() Hook

The useState() hook lets you declare state variables in a functional component. With useState, you can initialize a state variable and set a function to update its value. When the state updates, React will re-render the component with the new state value.

Syntax of useState:

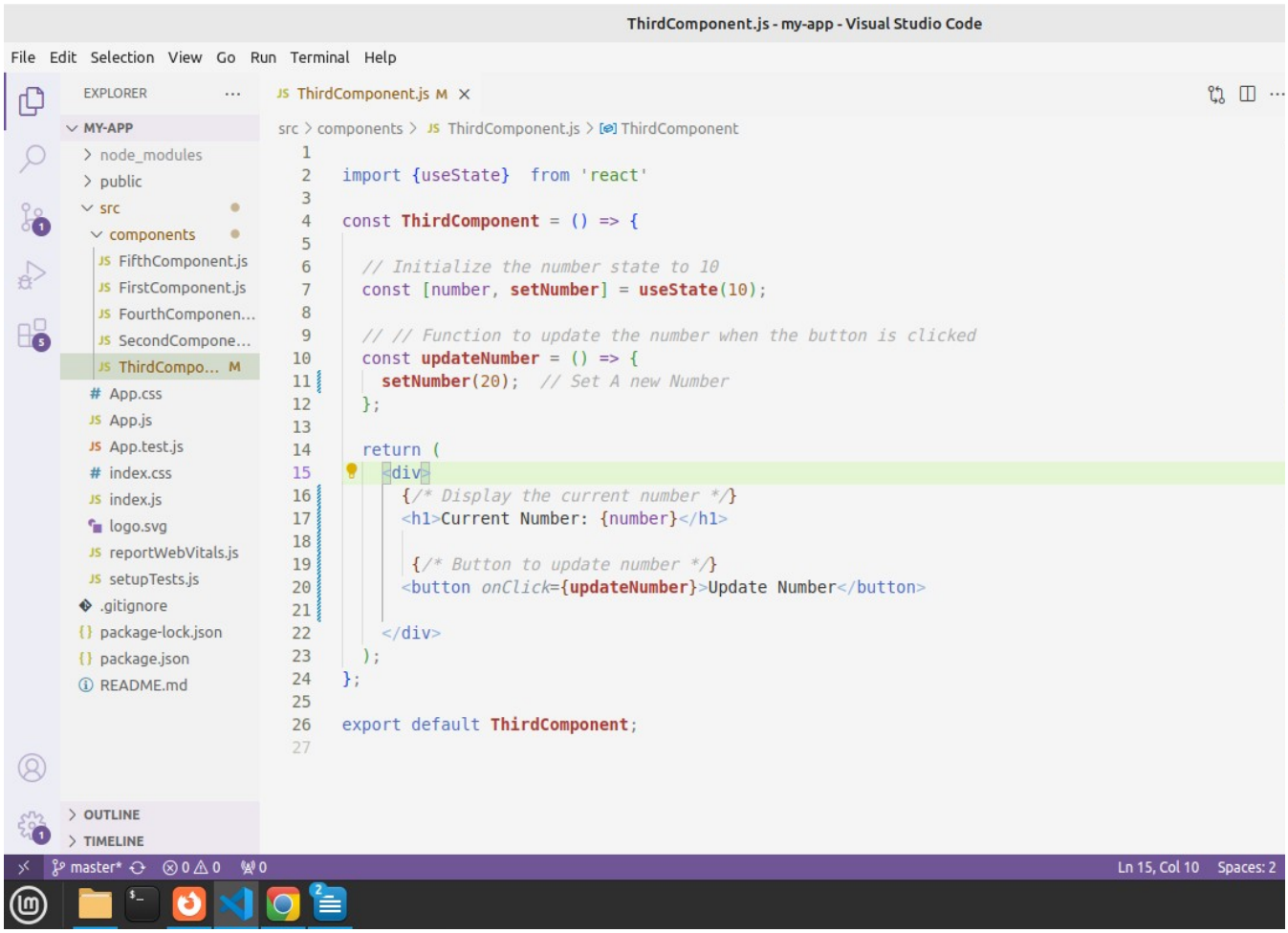
```
const [state, setState] = useState(initialValue);
```

- **state**: This is the current value of the state variable.
- **setState**: This is a function that you use to update the state variable.
- **initialValue**: This is the initial value of the state variable, which will be used when the component first renders.

2. Example of Using useState() in `ThirdComponent`

Let's use the **useState**() hook to manage a simple number state in our

Code: ThirdComponent.js - Functional Component



Render **ThirdComponent.js** in App.js to see the output

Explanation of ThirdComponent Using useState() Hook

Declare State Using useState():

- We initialize a state variable called number using **useState**(10). The 10 is the initial value of number. When the component first renders, number will be 10.
- setNumber is the function that allows us to **update the number** state.

Updating the State:

- Inside the **updateNumber** function, we use the setNumber function to

Update number to 20 each time the button is clicked.

- setNumber(20) updates the state with the new value.

Rendering the State:

- We use {number} inside the <h1> tag to display the current value of the state variable number in the UI. When the state is updated, React automatically re- renders the component to reflect the new value.

Button for Triggering State Update:

- We have a button with the text Update Number, and when it's clicked, it calls the **updateNumber** function. This function updates the state, and React re-renders the component.

4. How useState() Works

When the **`ThirdComponent`** component first renders:

Initial Rendering:

- number starts with the initial value of 10, and that is displayed on the screen inside the <h1> tag.

Button Click:

- When you click the Increase Number button, the **updateNumber** function is called.
- Inside **updateNumber**, **setNumber**(20) updates the state by 20

Re-Rendering:

- React re-renders the component with the updated state, so you see the new value of number on the screen.

For example:

- Initially, number = 10.
- After clicking the button once, number = 20

5. Why Use useState()?

- **Simplified State Management**: In functional components, useState() lets you add and manage states..
- **Reactivity**: When you call **setNumber**(), React knows that the state has changed, so it triggers a re-render of the component with the updated value.
- **Cleaner Code**: With hooks, you avoid writing more complex class-based life cycle methods and you can handle state management in a more concise way.

More practice

- Create below hooks using useState() hook and bind them in JSX
- marks, height, weight and points